

สร้าง Web App ที่เชื่อมต่อกับ Google Sheets ได้อย่างสมบูรณ์

เรียนรู้การส่งและดึงข้อมูลระหว่างหน้าเว็บกับฐานข้อมูล Google Sheets แบบ Real-time

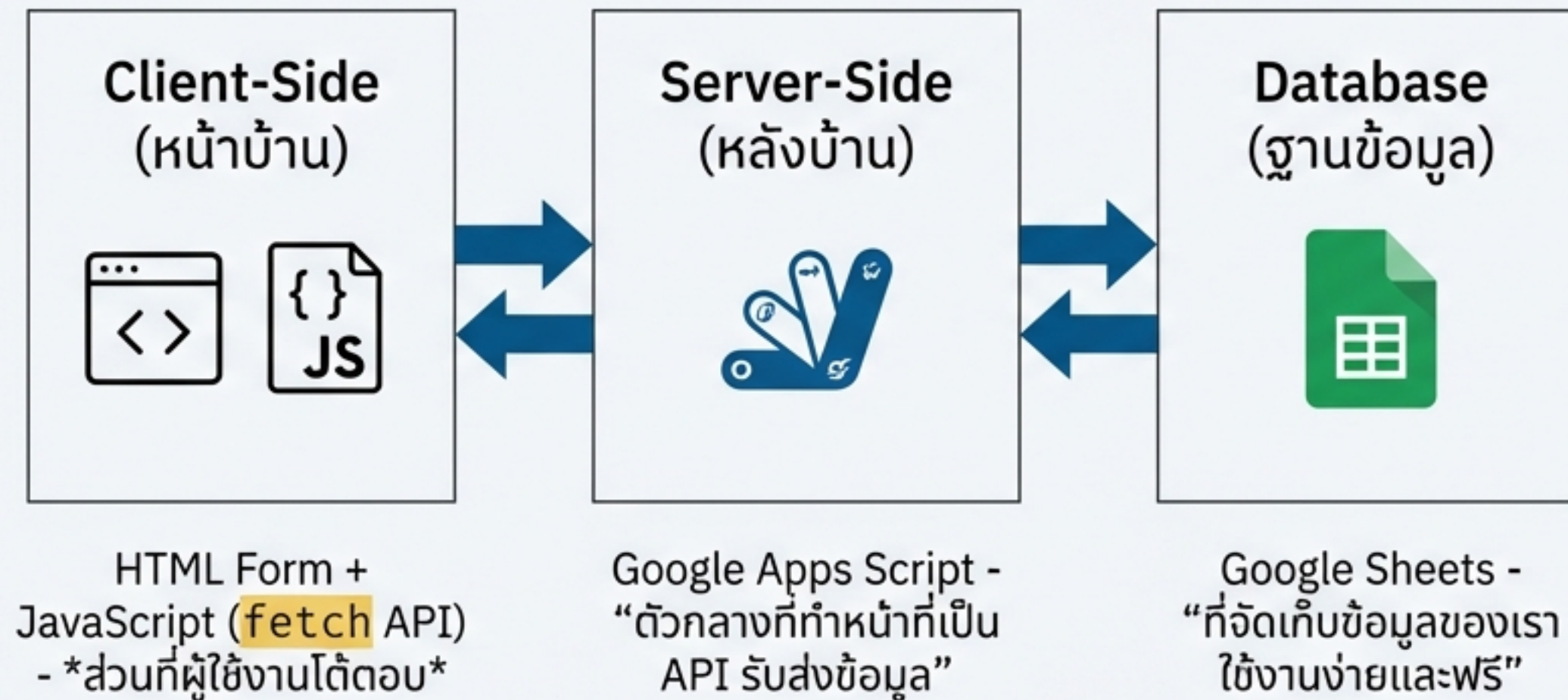


ใน Workshop นี้ เราจะสร้างระบบที่สมบูรณ์ตั้งแต่ต้นจนจบ

- **เป้าหมายที่ 1:** สร้างฟอร์มบนหน้าเว็บเพื่อรับข้อมูล แล้วส่งไปบันทึกที่ Google Sheets โดยอัตโนมัติ
- **เป้าหมายที่ 2:** ดึงข้อมูลทั้งหมดที่บันทึกไว้ใน Google Sheets กลับมาแสดงผลบนหน้าเว็บของเรา
- เมื่อจบ Workshop นี้ คุณจะสามารสรสร้าง Web App สำหรับเก็บและแสดงข้อมูลเบื้องต้นได้ด้วยตัวเอง

ภาพรวมสถาปัตยกรรมและเครื่องมือที่เราจะใช้

สถาปัตยกรรม / Architecture



เครื่องมือที่ต้องเตรียม / Tools



บัญชี Google
(Google Account)



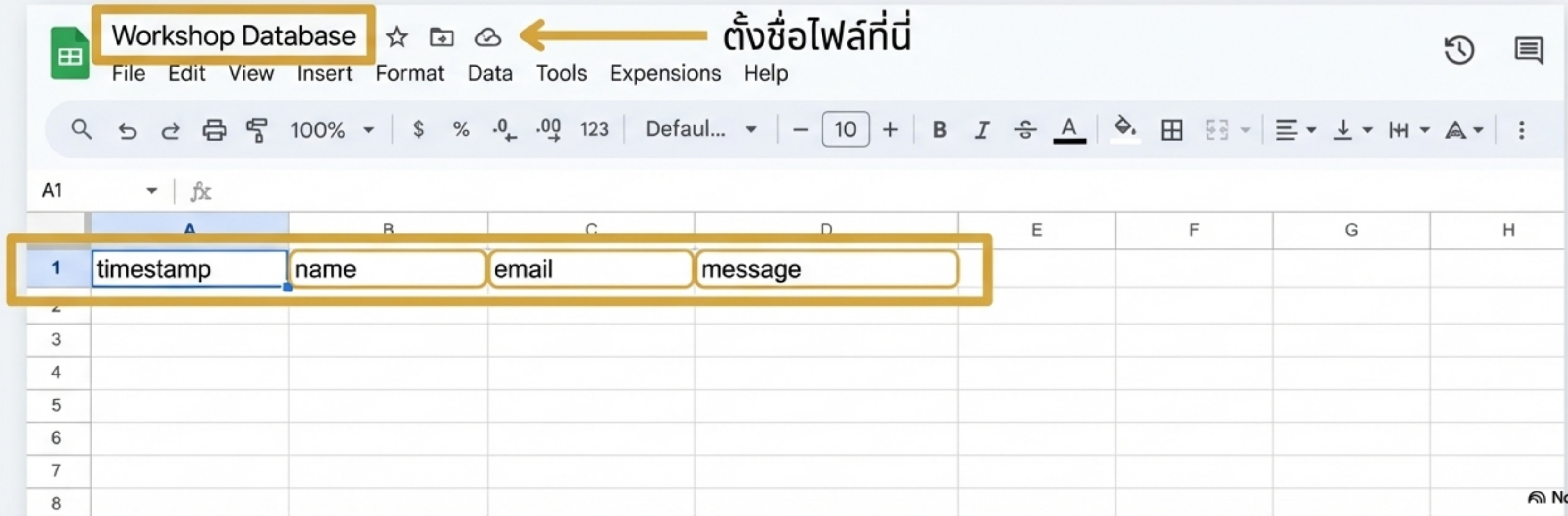
โปรแกรม Text Editor
(เช่น VS Code, Sublime Text, หรือ Notepad++)



เว็บเบราว์เซอร์
(แนะนำ Google Chrome)

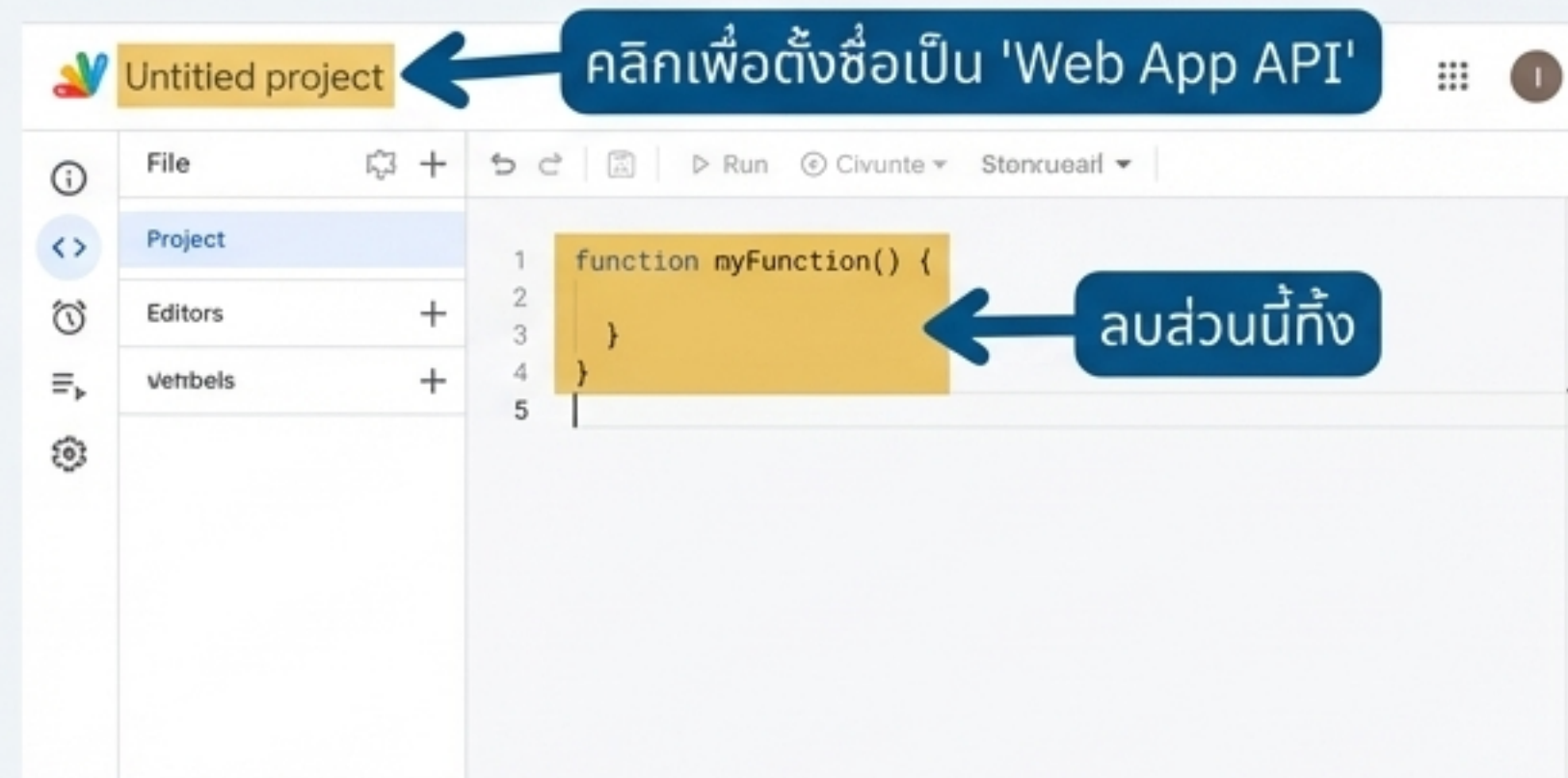
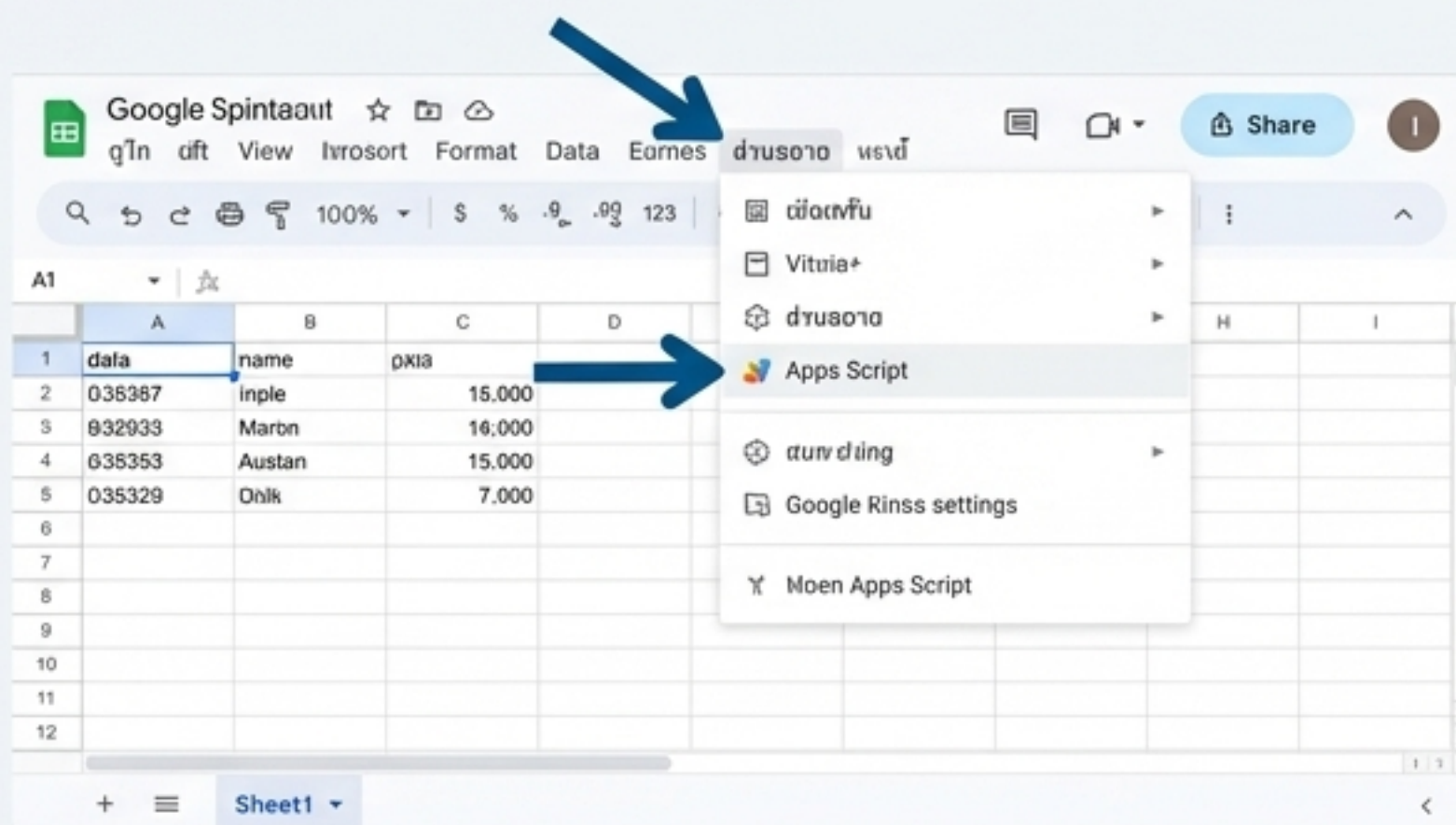
ขั้นตอนที่ 1: เตรียม Google Sheets สำหรับเป็นฐานข้อมูล

1. ไปที่ sheets.google.com และสร้าง Spreadsheet ใหม่
2. ตั้งชื่อไฟล์ว่า "Workshop Database" หรือชื่ออื่นตามต้องการ
3. ในแถวแรก (Row 1) ให้สร้างหัวข้อมูลคอลัมน์ (Headers) ที่เราจะใช้เก็บข้อมูล เช่น `timestamp`, `name`, `email`, `message`



ขั้นตอนที่ 2: สร้างโปรเจกต์ Google Apps Script

1. จากหน้า Google Sheets ที่เปิดอยู่ ไปที่เมนู **ส่วนขยาย (Extensions) > Apps Script**
2. ระบบจะเปิดหน้าต่าง Apps Script Editor ขึ้นมาในแท็บใหม่
3. ตั้งชื่อโปรเจกต์เป็น "Web App API"
4. ลบโค้ด `function myFunction() {}` ที่มีมาให้ทิ้งไป เราจะเริ่มเขียนโค้ดของเราเอง



เป็นฟังก์ชันพิเศษของ Google Apps Script ที่ทำงานโดยอัตโนมัติเมื่อมี HTTP POST request ส่งเข้ามาที่ URL ของ Web App ของเรา
`e` คือ object ที่เก็บข้อมูลทั้งหมดที่ส่งมา, รวมถึงข้อมูลจากฟอร์มใน `e.parameter`

ขั้นตอนที่ 3: เขียนฟังก์ชัน `doPost` เพื่อรับข้อมูลจากหน้าเว็บ

File: `Code.gs`

```
const SHEET_ID = "[YOUR_GOOGLE_SHEET_ID]"; // Use this Sheet ID from your Google Sheet's URL
const SHEET_NAME = "Sheet1"; // Or your specific sheet name

function doPost(e) {
  const sheet = SpreadsheetApp.openById(SHEET_ID).getSheetByName(SHEET_NAME);

  const newRow = [
    new Date(),
    e.parameter.name,
    e.parameter.email,
    e.parameter.message
  ];

  sheet.appendRow(newRow);

  return ContentService
    .createdTextOutput(JSON.stringify({ "status": "success", "data": JSON.stringify(newRow) }))
    .setMimeType(ContentService.MimeType.JSON);
}
```

⚠️ อย่าลืม! แก้ไข `SHEET\_ID` ให้เป็น ID ของ Sheet ของคุณเอง สามารถคัดลอกได้จาก URL

ทำความเข้าใจโค้ด `doPost` แต่ละส่วน

```
function doPost(e) {  
  const sheet = SpreadsheetApp.openById(SHEET_ID)  
    .getSheetByName(SHEET_NAME);  
  
  const newRow = [  
    new Date(),  
    e.parameter.name,  
    e.parameter.email,  
    e.parameter.message  
  ];  
  
  sheet.appendRow(newRow);  
  
  return ContentService  
    .createTextOutput(JSON.stringify({  
      "status": "success",  
      "data": JSON.stringify(newRow)  
    })))  
    .setMimeType(ContentService.MimeType.JSON);  
}
```

1

1. `const sheet = ...`: เชื่อมต่อกับ Google Sheet เป้าหมายของเราโดยใช้ ID และชื่อชีต

2

2. `const newRow = [...]`: สร้าง Array หรือแถวข้อมูลใหม่ โดยดึงค่าจาก object `e.parameter` ที่ถูกส่งมาจากฟอร์ม ค่าเหล่านี้จะตรงกับ `name` attribute ใน input HTML ของเรา

3

3. `sheet.appendRow(newRow)`: คำสั่งสำคัญในการเพิ่มแถวข้อมูลใหม่ที่เรารวสร้างขึ้นต่อท้ายข้อมูลที่มีอยู่

4

4. `return ContentService...`: ส่วนของการส่งคำตอบกลับไปยังหน้าเว็บ (Client) เพื่อบอกว่า "การบันทึกสำเร็จ" เราจะส่งข้อมูลกลับไปในรูปแบบ JSON ซึ่งเป็นมาตรฐานสากล

ขั้นตอนที่ 4: สร้างหน้าเว็บด้วย HTML สำหรับกรอกข้อมูล

สร้างไฟล์ใหม่ใน Text Editor ของคุณ ตั้งชื่อว่า `index.html` แล้วใส่โค้ดนี้

File: `index.html`

```
<!DOCTYPE html>
<html>
  <head>
    <title>Data Form</title>
  </head>
  <body>
    <h1>ส่งข้อมูลไปยัง Google Sheets</h1>
    <form id="data-form">
      <input type="text" name="name" placeholder="ชื่อ" required><br><br>
      <input type="email" name="email" placeholder="อีเมล" required><br><br>
      <textarea name="message" placeholder="ข้อความ" required></textarea><br><br>
      <button type="submit">ส่งข้อมูล</button>
    </form>
    <div id="status"></div>
  </body>
</html>
```

FYI

สังเกต `name` attribute ใน code: <input> (เช่น `name="name"`, `name="email"`) คำ `name` นี้จะต้องตรงกับที่เราใช้ดึงข้อมูลใน Apps Script (`e.parameter.name`, `e.parameter.email`) ทุกประการ

ขั้นตอนที่ 5: เขียน JavaScript เพื่อส่งข้อมูลฟอร์ม

เพิ่มโค้ด ``<script>`` นี้เข้าไปก่อนปิด tag ``</body>`` ในไฟล์ ``index.html`

File: ``index.html`` – inside `<script>` tag

```
const form = document.getElementById('data-form');
const statusDiv = document.getElementById('status');
const SCRIPT_URL = "[YOUR_WEB_APP_URL]"; // เราจะได้ URL นี้ในขั้นตอนต่อไป

form.addEventListener('submit', e => {
  e.preventDefault(); // ป้องกันไม่ให้หน้าเว็บรีเฟรช

  statusDiv.innerHTML = "กำลังส่งข้อมูล...";
  fetch(SCRIPT_URL, {
    method: 'POST',
    body: new FormData(form)
  })
  .then(response => response.json())
  .then(data => {
    console.log('Success:', data);
    statusDiv.innerHTML = "ส่งข้อมูลสำเร็จ!";
    form.reset();
  })
  .catch(error => {
    console.error('Error!', error.message);
    statusDiv.innerHTML = "เกิดข้อผิดพลาด!";
  });
});
```

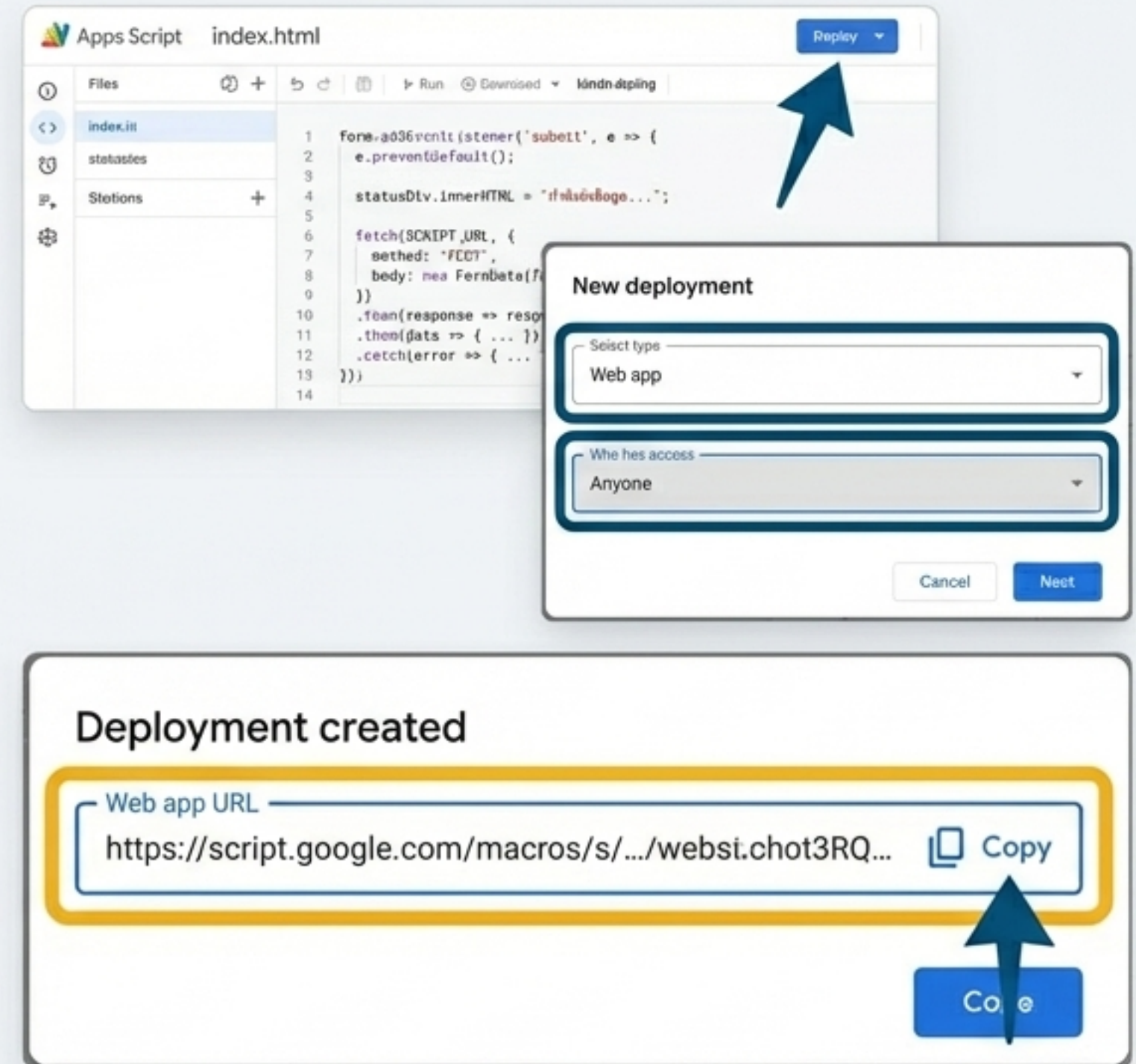

ทำความเข้าใจโค้ด JavaScript สำหรับส่งข้อมูล

```
1 form.addEventListener('submit', e => {  
2   e.preventDefault();  
3  
4   statusDiv.innerHTML = "กำลังส่งข้อมูล...";  
5  
6   fetch(SCRIPT_URL, {  
7     method: 'POST',  
8     body: new FormData(form)  
9   })  
10  .then(response => response.json())  
11  .then(data => { ... })  
12  .catch(error => { ... });  
13 });
```

- 1 **e.preventDefault()**: หยุดพฤติกรรมปกติของฟอร์ม (ที่จะเปลี่ยนหน้าหรือรีเฟรช) เพื่อให้เราควบคุมการส่งข้อมูลด้วย JavaScript ได้เอง
- 2 **fetch(SCRIPT_URL, ...)**: คือหัวใจหลักของการสื่อสาร เป็น Web API ของเบราว์เซอร์สำหรับส่ง HTTP Request ไปยังเซิร์ฟเวอร์ (Apps Script ของเรา)
- 3 **method: 'POST'**: ระบุว่าเรากำลัง "ส่ง" ข้อมูล ซึ่งจะไปที่ฟังก์ชัน `doPost` ที่เราเขียนไว้
- 4 **body: new FormData(form)**: รวบรวมข้อมูลทั้งหมดจากฟอร์มของเราเพื่อเตรียมส่ง
- 5 **.then(...)**: ทำงานเมื่อ `fetch` ได้รับการตอบกลับจากเซิร์ฟเวอร์สำเร็จ ใช้สำหรับจัดการกับข้อมูลที่ส่งกลับมา
- 6 **.catch(...)**: ทำงานเมื่อเกิดข้อผิดพลาดในการเชื่อมต่อ

ขั้นตอนที่ 6: Deploy Apps Script เพื่อให้ได้ URL

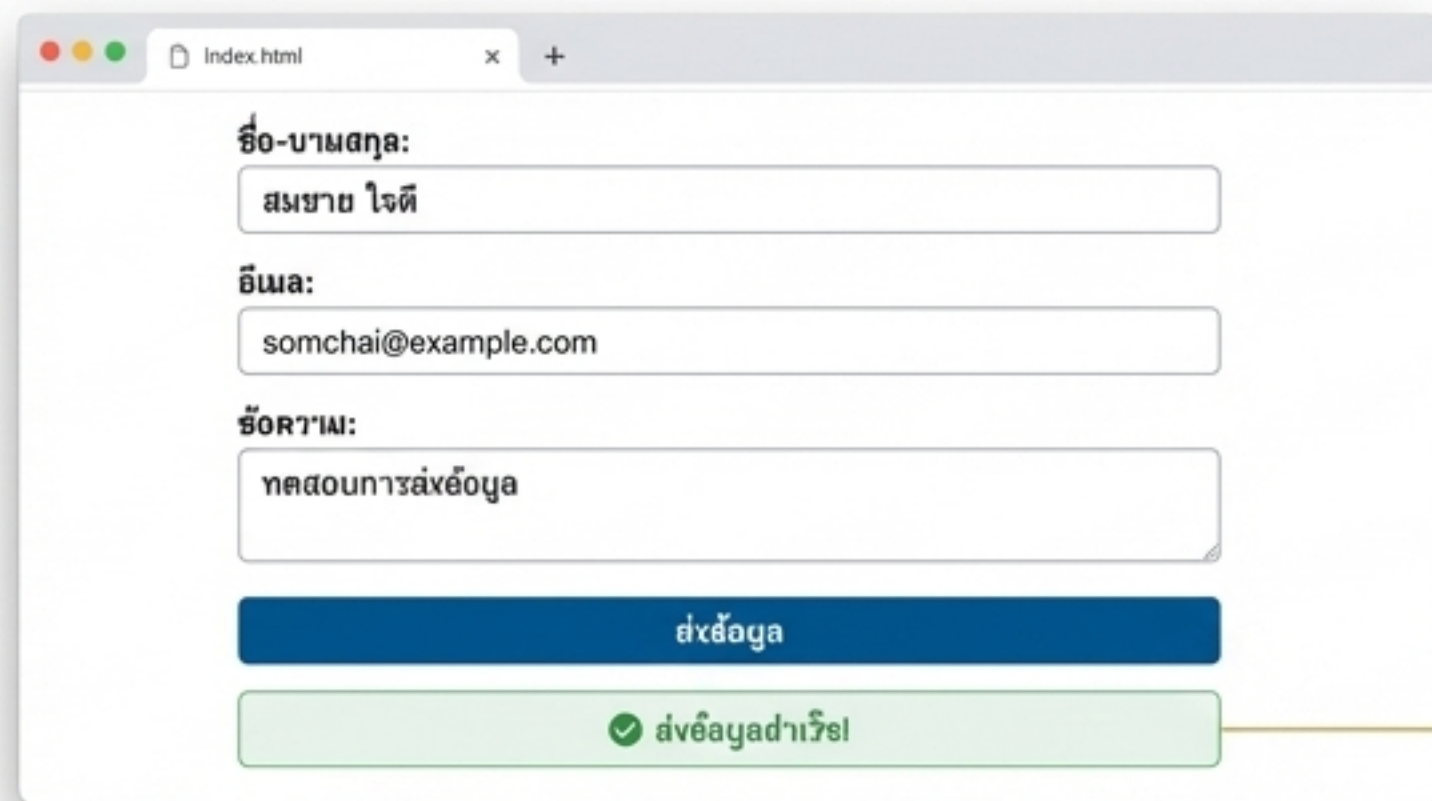
1. กลับไปที่หน้า Apps Script Editor
2. คลิกที่ปุ่มสีฟ้า **"การทำให้ใช้งานได้" (Deploy) > "การทำให้ใช้งานได้รายการใหม่"** (New deployment)
3. เลือกประเภทเป็น **"เว็บแอป"** (Web app)
4. ในส่วน **"ผู้ที่มีสิทธิ์เข้าถึง"** (Who has access) ให้เลือกเป็น **"ทุกคน"** (Anyone)
5. คลิก **"ทำให้ใช้งานได้"** (Deploy)
6. **สำคัญมาก:** คัดลอก **"URL ของเว็บแอป"** (Web app URL) ที่ได้มาเก็บไว้



ขั้นตอนที่ 7: ทดสอบการส่งข้อมูล!

1. นำ Web App URL ที่คัดลอกมาจากขั้นตอนที่แล้ว ไปวางในตัวแปร SCRIPT_URL ในไฟล์ `index.html` ของคุณ
2. เปิดไฟล์ `index.html` ด้วยเว็บเบราว์เซอร์
3. ลองกรอกข้อมูลในฟอร์มแล้วกด "ส่งข้อมูล"

ผลลัพธ์ (Result)



Index.html

ชื่อ-นามสกุล:

สมชาย ใจดี

อีเมล:

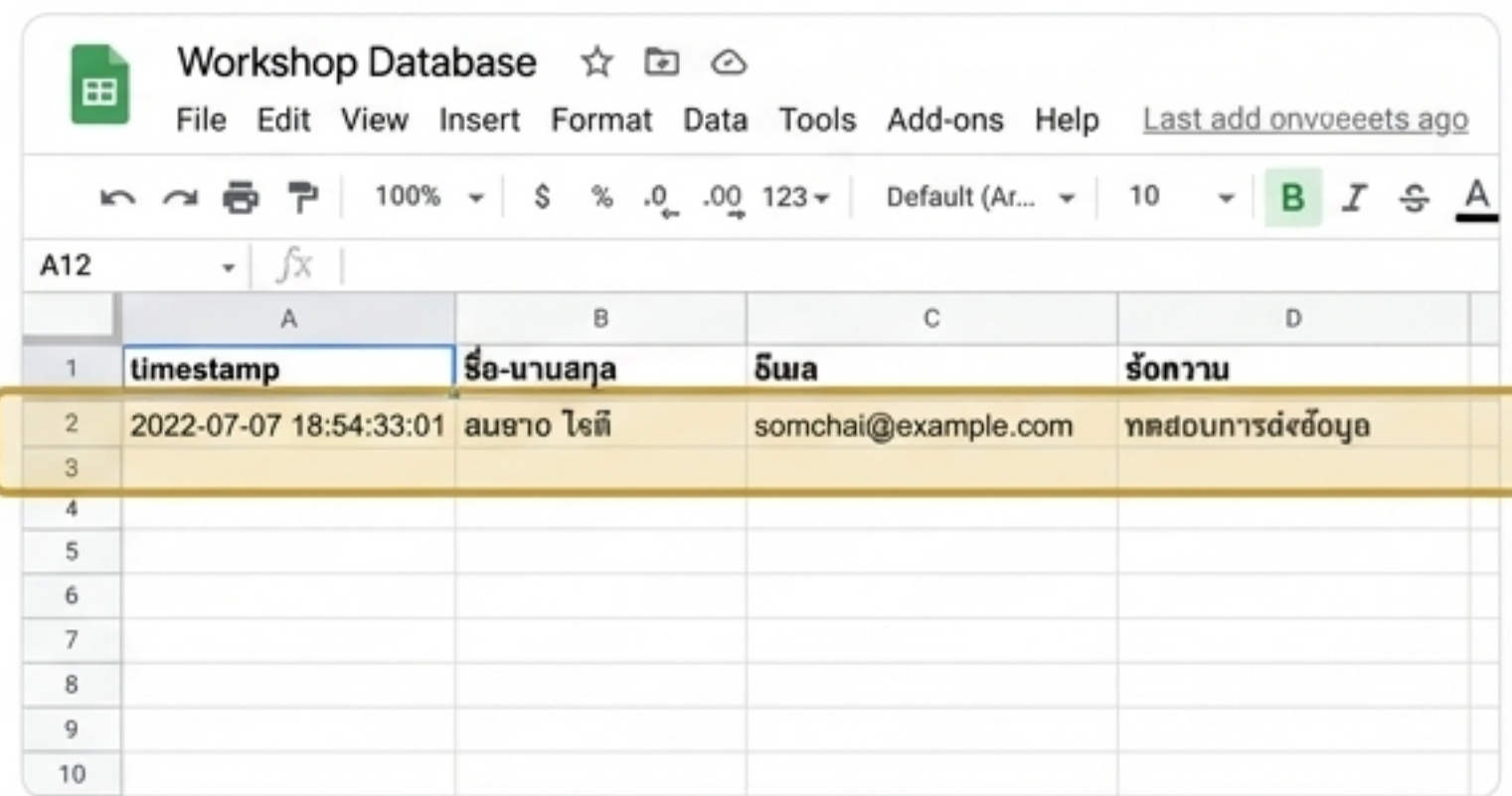
somchai@example.com

ข้อความ:

ทดสอบการส่งข้อมูล

ส่งข้อมูล

✓ ส่งข้อมูลสำเร็จ!



Workshop Database

File Edit View Insert Format Data Tools Add-ons Help Last add on vooeets ago

	A	B	C	D
1	timestamp	ชื่อ-นามสกุล	อีเมล	ใช้งาน
2	2022-07-07 18:54:33:01	สมชาย ใจดี	somchai@example.com	ทดสอบการส่งข้อมูล
3				
4				
5				
6				
7				
8				
9				
10				

ส่วนที่ 2: ดึงข้อมูลจาก Google Sheets มาแสดงผล

ขั้นตอนที่ 8: เขียนฟังก์ชัน **`doGet`** ใน Apps Script

`doGet(e)` คืออะไร?

เช่นเดียวกับ **`doPost`**, **`doGet`** เป็นฟังก์ชันพิเศษที่จะทำงานเมื่อมี HTTP GET request ส่งเข้ามา ซึ่งโดยปกติคือการเรียก URL ของ Web App ตรงๆ ผ่านเบราว์เซอร์หรือ **`fetch`** เพื่อ "ขอ" ข้อมูล

File: `Code.gs` - เพิ่มต่อท้ายโค้ดเดิม

```
function doGet(e) {  
  const sheet = SpreadsheetApp.openById(SHEET_ID).getSheetByName(SHEET_NAME);  
  const data = sheet.getDataRange().getValues();  
  const headers = data.shift(); // Get and remove header row  
  
  const jsonData = data.map(row => {  
    let obj = {};  
    headers.forEach((header, index) => {  
      obj[header] = row[index];  
    });  
    return obj;  
  });  
  
  return ContentService  
    .createTextOutput(JSON.stringify({ "status": "success", "data": jsonData } ))  
    .setMimeType(ContentService.MimeType.JSON);  
}
```


ทำความเข้าใจโค้ด `doGet`

```
function doGet(e) {  
  const sheet = SpreadsheetApp.openById(SHEET_ID).getSheetByName(SHEET_NAME);  
  const data = sheet.getDataRange().getValues();  
  const headers = data.shift();  
  
  const jsonData = data.map(row => {  
    let obj = {};  
    headers.forEach((header, index) => {  
      obj[header] = row[index];  
    });  
    return obj;  
  });  
  
  return ContentService  
    .createTextOutput(JSON.stringify({ "status": "success",  
      "data": jsonData }))  
    .setMimeType(ContentService.MimeType.JSON);  
}
```

1

sheet.getDataRange().getValues(): ดึงข้อมูลทั้งหมดในชีตออกมาในรูปแบบ 2D Array (Array ช้อน Array)

2

const headers = data.shift(): ดึงแถวแรกสุด (หัวตาราง) ออกมาเก็บไว้ในตัวแปร 'headers' และลบออกจาก 'data' เติม

3

data.map(row => {...}): วนลูปข้อมูลแต่ละแถว ('row') เพื่อแปลงจาก Array (['John Doe', 'john@email.com', ...]) ให้เป็น JSON Object ({name: 'John Doe', email: 'john@email.com', ...}) โดยใช้ 'headers' เป็น key ซึ่งเป็นรูปแบบที่ JavaScript ฝั่งหน้าบ้านจะทำงานด้วยง่ายกว่ามาก

4

return ContentService...: ส่งข้อมูลทั้งหมดที่แปลงแล้วกลับไปในรูปแบบ JSON

ขั้นตอนที่ 9: อัปเดตหน้าเว็บเพื่อแสดงข้อมูล

1. ในไฟล์ `index.html` เพิ่ม `<div id="data-container"></div>` และ `<button id="load-data">Load Data</button>`
2. ในส่วน `<script>` เพิ่มฟังก์ชัน `loadData()`

File: `index.html` - อัปเดต HTML

```
<!-- ... form ... -->
<hr>
<h2>ข้อมูลที่บันทึกไว้</h2>
<button id="load-data">ดึงข้อมูลล่าสุด</button>
<div id="data-container"></div>
```

File: `index.html` - อัปเดต Script

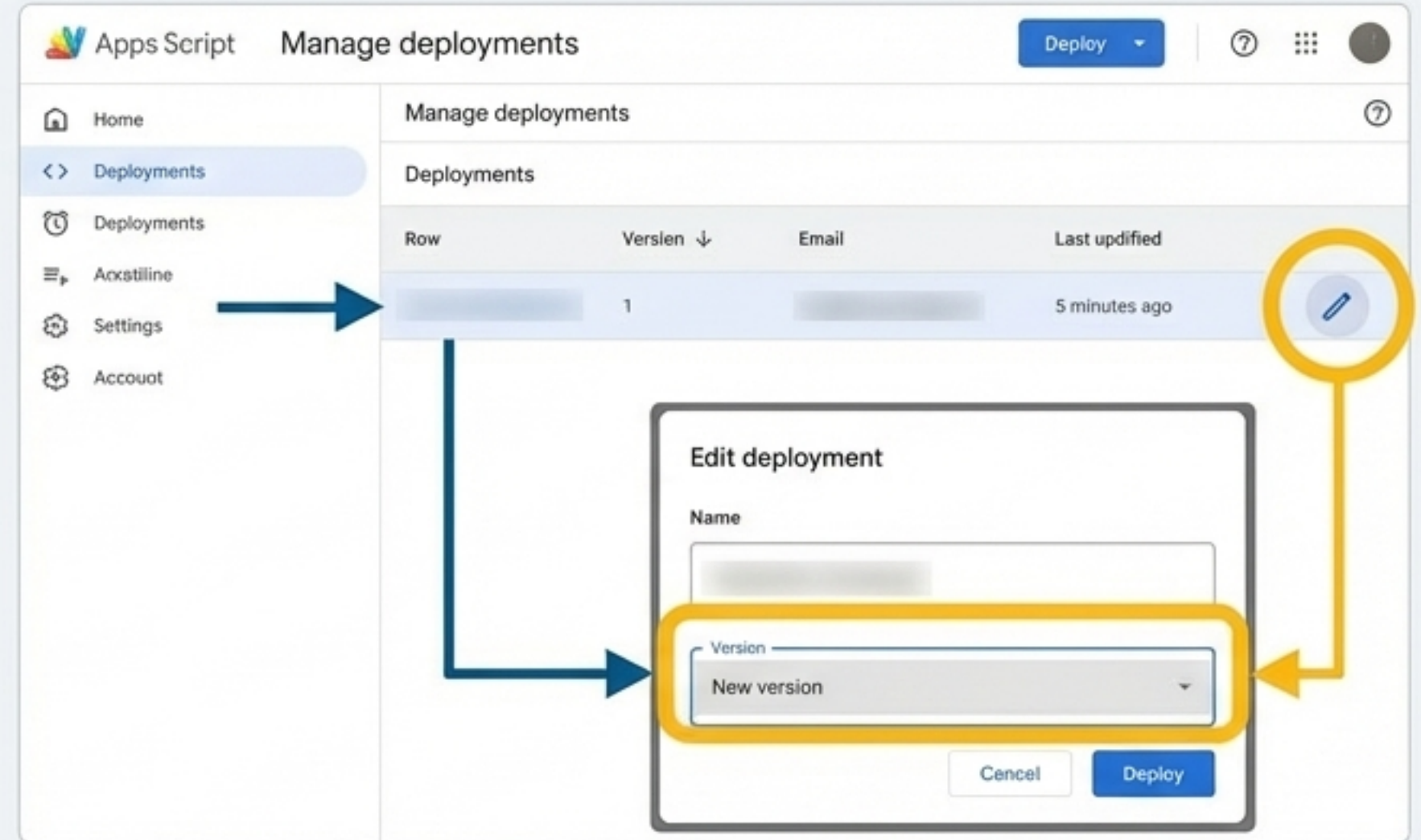
```
// ... add this inside the <script> tag ...
const dataContainer = document.getElementById('data-container');
const loadBtn = document.getElementById('load-data');

function loadData() {
  dataContainer.innerHTML = 'กำลังโหลดข้อมูล...';
  fetch(SCRIPT_URL) // GET request is the default
    .then(res => res.json())
    .then(body => {
      // Assuming body.data is an array of objects
      let html = '<table><tr><th>Name</th><th>Email</th><th>Message</th></tr>';
      body.data.forEach(row => {
        html += `<tr><td>${row.name}</td><td>${row.email}</td><td>${row.message}</td></tr>`;
      });
      html += '</table>';
      dataContainer.innerHTML = html;
    });
}

loadBtn.addEventListener('click', loadData);
```


ขั้นตอนที่ 10: Deploy ซ้ำอีกครั้ง (สำคัญ!)

- ทุกครั้งที่มีการแก้ไขโค้ดในฝั่ง Apps Script (`Code.gs`) เราจะต้อง Deploy ใหม่เพื่อให้การเปลี่ยนแปลงมีผล
- กลับไปที่ Apps Script Editor
- คลิก "การทำให้ใช้งานได้" (Deploy) > "จัดการการทำให้ใช้งานได้" (Manage deployments)
- เลือก Deployment ที่ใช้งานอยู่ แล้วคลิกไอคอนรูปดินสอ (แก้ไข)
- ในช่อง "เวอร์ชัน" ให้เลือก "เวอร์ชันใหม่" (New version)
- คลิก "ทำให้ใช้งานได้" (Deploy)

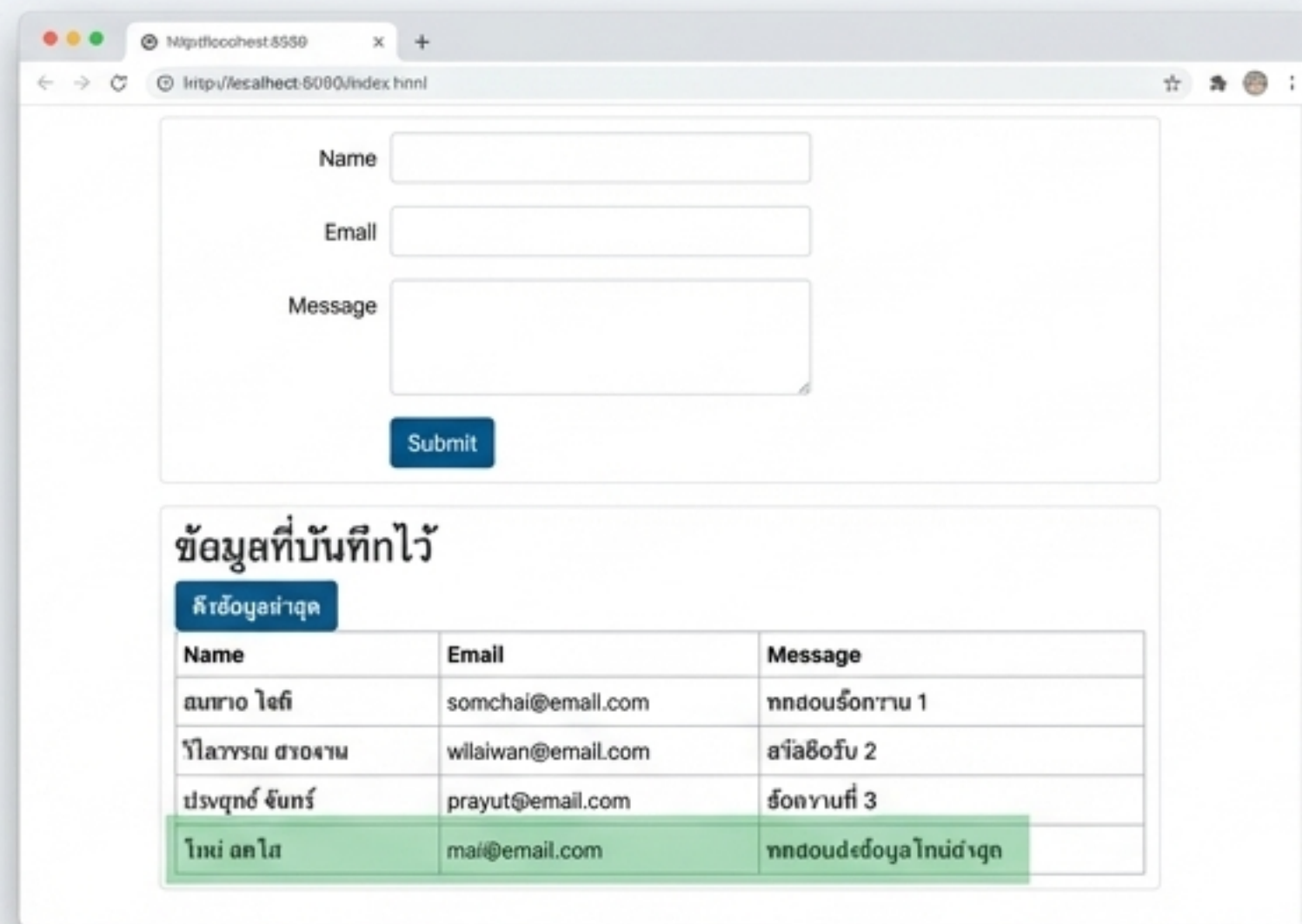


FYI: ทำไมต้อง Deploy ใหม่?

เพราะ Web App จะทำงานจากโค้ดเวอร์ชันที่เรา Deploy ไว้เท่านั้น การแค่กดบันทึก (Save project) จะไม่ทำให้ Web App ที่ออนไลน์อยู่เปลี่ยนแปลงตามไปด้วย

ขั้นตอนที่ 11: ทดสอบระบบทั้งหมด

1. รีเฟรชหน้า `index.html` ในเบราว์เซอร์ของคุณ
2. คลิกปุ่ม "**ดึงข้อมูลล่าสุด**" > คุณจะเห็นข้อมูลทั้งหมดจาก Google Sheets
ถูกโหลดมาแสดงเป็นตาราง
3. ลองเพิ่มข้อมูลใหม่ผ่านฟอร์ม
4. เมื่อส่งสำเร็จ ให้กดปุ่ม "**ดึงข้อมูลล่าสุด**"
อีกครั้ง > ข้อมูลใหม่ที่คุณเพิ่งเพิ่มเข้า
ไปควรจะปรากฏในตารางทันที



The screenshot shows a web browser window with a contact form and a table of data. The form has fields for Name, Email, and Message, with a Submit button. Below the form is a table titled 'ข้อมูลที่บันทึกไว้' (Recorded Data) with columns for Name, Email, and Message. The table contains five rows of data, with the last row highlighted in green.

Name	Email	Message
สมชาย ใจดี	somchai@email.com	ททตตตตตตตต 1
วิลัยวรรณ งามงาม	wilaiwan@email.com	สวัสดีครับ 2
ประยุทธ์ จันทร	prayut@email.com	สวัสดีครับ 3
ไมเคิล ดน	mai@email.com	ททตตตตตตตต ไมเคิล

การจัดการ Error เบื้องต้น (Error Handling)

ทำให้แอปพลิเคชันของเราทนทานต่อข้อผิดพลาดมากขึ้น

ฝั่ง Server: `Code.gs`

ใช้ `try...catch` block เพื่อดักจับข้อผิดพลาดที่อาจเกิดขึ้น เช่น ชีตหาไม่เจอ, เขียนข้อมูลไม่สำเร็จ

```
function doPost(e) {  
  try {  
    // ... original code to append row ...  
    return ContentService.createTextOutput(  
      JSON.stringify({ "status": "success" })...  
    } catch (error) {  
      return ContentService.createTextOutput(  
        JSON.stringify({ "status": "error",  
          "message": error.message }))....  
    }  
  }  
}
```

ฝั่ง Client: `index.html`

ใช้ `.catch()` ของ `fetch` เพื่อจัดการกับปัญหา network หรือ server error และตรวจสอบ `status` ที่ส่งกลับมาจาก Server

```
.then(response => response.json())  
.then(data => {  
  if (data.status === 'success') {  
    statusDiv.innerHTML = "ส่งข้อมูลสำเร็จ!";  
  } else {  
    statusDiv.innerHTML = "เกิดข้อผิดพลาด: " +  
data.message;  
  }  
})  
.catch(error => ...);
```


แนวคิดการนำไปประยุกต์ใช้งานจริง (Real-world Applications)



ระบบฟอร์มลงทะเบียน: สร้างฟอร์มสำหรับลงทะเบียนเข้าร่วมกิจกรรมหรืองานอีเวนต์



แบบสำรวจความคิดเห็น (Survey Form): สร้างแบบสอบถามออนไลน์และเก็บข้อมูลผลลัพธ์อัตโนมัติ



ระบบ Contact Form สำหรับเว็บไซต์: เพิ่มฟอร์ม "ติดต่อเรา" ในเว็บไซต์ส่วนตัวหรือของบริษัท



เครื่องมือเก็บข้อมูลส่วนตัว: สร้างแอปง่ายๆ สำหรับจดบันทึกค่าใช้จ่าย, ติดตามการออกกำลังกาย, หรือเก็บ Log ข้อมูลต่างๆ



Dashboard ข้อมูลเบื้องต้น: สร้างหน้าเว็บสำหรับแสดงผลข้อมูลจาก Sheet แบบ Real-time

สรุปแนวคิดและฟังก์ชันหลักที่ได้เรียนรู้

Google Apps Script (Server-Side)

- `doPost(e)` : รับข้อมูลที่ส่งมาแบบ POST
- `doGet(e)` : รับ request เพื่อส่งข้อมูลกลับไป
- `SpreadsheetApp` : Service หลักสำหรับจัดการ Google Sheets
- `ContentService` : สร้าง Output (เช่น JSON) เพื่อส่งกลับ
- ****Deployment****: การทำให้สคริปต์กลายเป็น Web App ที่เข้าถึงได้จากภายนอก

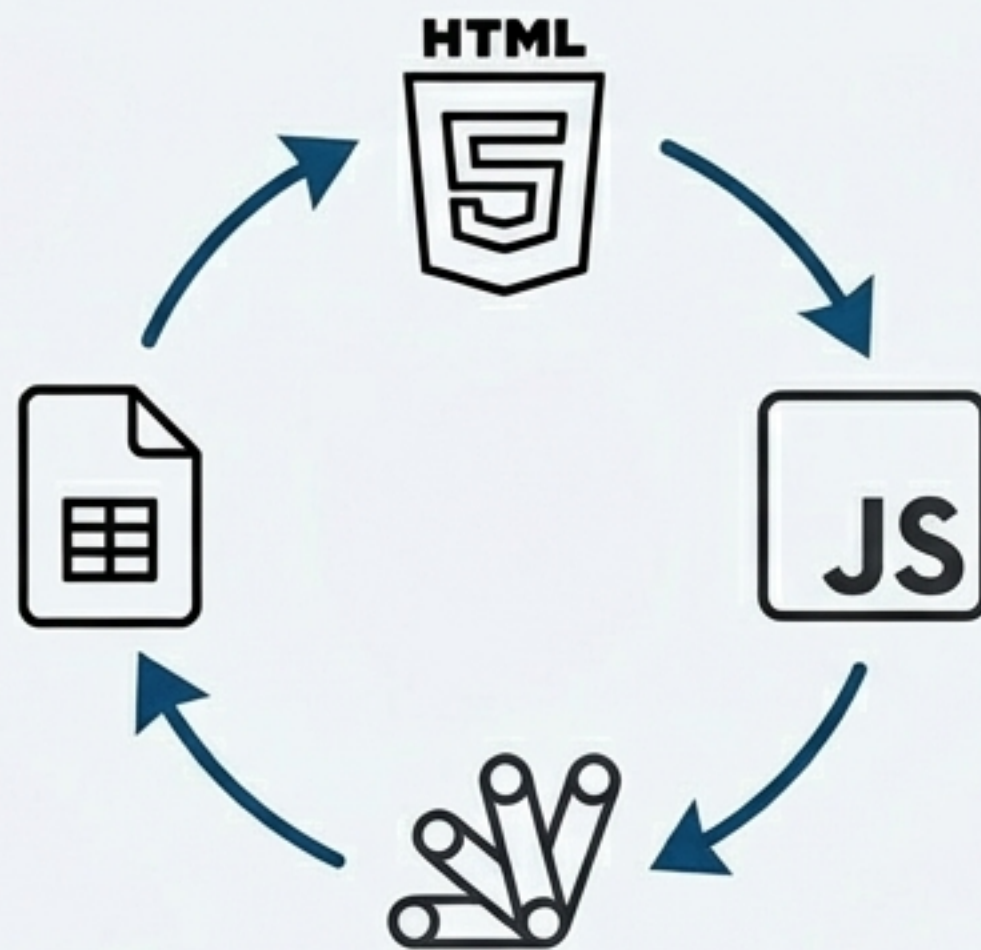
Web Page (Client-Side)

- `fetch()` : API สำหรับส่ง/รับข้อมูลกับ Server (Asynchronous)
- `new FormData()` : เครื่องมือสำหรับรวบรวมข้อมูลจากฟอร์ม
- `.preventDefault()` : หยุดพฤติกรรมดีฟอลต์ของอีเวนต์
- `.then()` & `.catch()` : การจัดการกับผลลัพธ์ (Success/Error) ของ Promise

Data Format

- JSON : รูปแบบข้อมูลมาตรฐานสำหรับการสื่อสารระหว่าง Client-Server

คุณได้สร้าง Full-Stack Web Application สำเร็จแล้ว



ระบบที่คุณสร้างขึ้นในวันนี้คือพื้นฐานที่แข็งแกร่งสำหรับการพัฒนา Web Application ที่ซับซ้อนยิ่งขึ้น
ลองนำความรู้ที่ได้ไปทดลองแก้ไข, เพิ่มฟังก์ชันใหม่ๆ, หรือสร้างโปรเจกต์ของตัวเอง

The possibilities are now in your hands. **Happy coding!**